

112. Lire la case devant soi : le mur

Avant de faire un pas, on regarde quelle tuile il y a là où l'on va.

1. Ce qu'il faut comprendre

Les murs ne sont pas des lutins : ils sont **dans le décor**, posés avec `poser()`. Pour savoir si le héros peut avancer, on ne compare donc pas des boîtes : on **lit la case** où il va entrer, avec `lire(colonne, ligne)` qui rend la tuile posée là.

Le héros compte en **pixels**, la grille en **cases**. Le pont entre les deux est la division de la leçon « au pixel près » : le pixel `p` est dans la case `p / 8`.

Ce qui est nouveau ici : lire la case du pixel qu'on va toucher. Le héros occupe les pixels `x` à `x + 7`. Un pas à droite le fait entrer dans le pixel `x + 8` : on lit la case `(x + 8) / 8`. Un pas à gauche, dans le pixel `x - 1` : on lit la case `(x - 1) / 8`.

Déroulons à droite. Le mur est en colonne 16, qui couvre les pixels 128 à 135. Héros en `x = 119` : son prochain pixel est 127, case $127 / 8 = 15$, vide — il avance. En `x = 120` : prochain pixel 128, case 16 : **le mur**. Il s'arrête, collé contre lui, sans le traverser d'un seul pixel.

À gauche, le mur est en colonne 3 (pixels 24 à 31). Le héros s'arrête en `x = 32` : son prochain pixel serait 31, dans le mur.

Le héros est posé sur la ligne de pixels $64 = 8 \times 8$: il est **pile** dans la ligne de cases 8. Il n'y a donc qu'une ligne de cases à regarder. La leçon suivante le libère de haut en bas.

À toi : pose un deuxième mur en colonne 10 et vérifie que le héros s'y arrête des deux côtés.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **673 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          65 o ← la console, à l'allumage — la mise en route de la console
main()             130 o ← démarrage — ta fonction
VBlank              8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage       28 o ← image() — attend l'image suivante
```

AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte	– attend de pouvoir écrire une case
AdresseCase	10 o	← lire()	– trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage	– range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage	– installe la copie des lutins en mémoire rapide
CopierTuiles	85 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	208 o	← CopierTuiles	– 11 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert	– la petite routine qui copie les lutins
"GAUCHE DROITE"	14 o	← texte()	– un texte gravé

Lettres gravées : ACDEGHIORTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

lire()	× 2	38 o sur place, dans main()	+ AdresseCase (10 o)
sprite()	× 1	23 o sur place, dans main()	
poser()	× 2	12 o sur place, dans main()	
texte()	× 1	11 o sur place, dans main()	+ EcrireTexte (10 o)
bouton()	× 2	10 o sur place, dans main()	+ LireManette (37 o)
image()	× 1	6 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

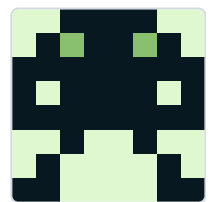
```
// CE PROGRAMME : un héros va à gauche et à droite entre deux murs,
// et s'arrête collé contre eux, sans les traverser.
//
// Ce qui est nouveau : lire la case du décor où l'on va entrer.
// lire(colonne, ligne) rend la tuile posée dans cette case.
// Le héros compte en PIXELS, le décor en CASES : le pixel p est dans la case p / 8.
// Le héros occupe les pixels x à x + 7 :
// un pas à droite le fait entrer dans le pixel x + 8 → la case (x + 8) / 8 ;
// un pas à gauche dans le pixel x - 1 → la case (x - 1) / 8.

#include <Tuile> // un dessin de 8 × 8 pixels
#include <texte> // écrit un texte à l'écran
#include <poser> // pose une tuile sur une case du fond
#include <bouton> // lit un bouton de la manette
#include <lire> // lit la tuile posée sur une case
#include <sprite> // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — HEROS

```
Tuile HEROS = {
    "..####..",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "##...##",
};

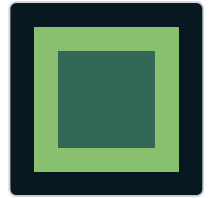
// Un bloc de mur, dessiné en quatre nuances.
```



à l'écran

3. UN DESSIN — MUR

```
Tuile MUR = {
  "33333333",
  "31111113",
  "31222213",
  "31222213",
  "31222213",
  "31222213",
  "31111113",
  "33333333",
};
```



à l'écran

4. UNE VARIABLE — X

```
uint8_t x = 76;      // en pixels ; le héros occupe les pixels x à x + 7
```

5. UNE VARIABLE — Y

```
uint8_t y = 64;      // 64 = 8 x 8 : pile sur la ligne de cases 8
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {          // Le jeu commence ici.
  texte(1, 1, "GAUCHE DROITE");
  poser(3, 8, MUR);    // colonne 3 : les pixels 24 à 31
  poser(16, 8, MUR);   // colonne 16 : les pixels 128 à 135

  while (true) {       // La boucle du jeu :
    image();

    // À droite : le prochain pixel est x + 8, dans la case (x + 8) / 8.
    // x = 119 : pixel 127, case 15, vide → il avance.
    // x = 120 : pixel 128, case 16 : le MUR → il s'arrête.
    if (bouton(DROITE) && lire((x + 8) / 8, 8) != MUR) x++;

    // À gauche : le prochain pixel est x - 1, dans la case (x - 1) / 8.
    // x = 32 : pixel 31, case 3 : le MUR → il s'arrête.
    if (bouton(GAUCHE) && lire((x - 1) / 8, 8) != MUR) x--;

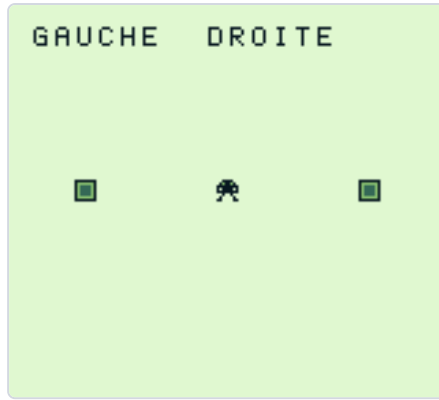
    sprite(0, x, y, HEROS); // On pose le héros à sa place.
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



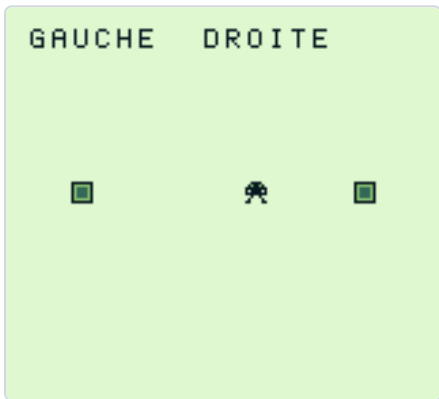
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Un héros entre deux blocs de mur ; il avance jusqu'à les toucher, et s'arrête pile contre eux.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ à droite, il s'arrête contre le mur : $x = 120$ (120)

✓ à gauche aussi : $x = 32$ (32)

✓ les murs sont toujours là

Ensuite : **113. Glisser le long d'un mur : un axe à la fois** — chapitre 18 : Les collisions : se toucher, se cogner. · gameboy3 — écrire une cartouche Game Boy en C++